Random Forest Project

For this project we will be exploring publicly available data from LendingClub.com. Lending Club connects people who need money (borrowers) with people who have money (investors). Hopefully, as an investor you would want to invest in people who showed a profile of having a high probability of paying you back. We will try to create a model that will help predict this.

Lending club had a very interesting year in 2016, so let's check out some of their data and keep the context in mind. This data is from before they even went public.

We will use lending data from 2007-2010 and be trying to classify and predict whether or not the borrower paid back their loan in full. You can use the csv already provided on Blackboard. The csv provided has been cleaned of NA values.

Here are what the columns represent:

- credit.policy: 1 if the customer meets the credit underwriting criteria of LendingClub.com, and 0 otherwise.
- purpose: The purpose of the loan (takes values "credit_card", "debt_consolidation", "educational", "major_purchase", "small_business", and "all_other").
- int.rate: The interest rate of the loan, as a proportion (a rate of 11% would be stored as 0.11). Borrowers judged by LendingClub.com to be more risky are assigned higher interest rates.
- installment: The monthly installments owed by the borrower if the loan is funded.
- log.annual.inc: The natural log of the self-reported annual income of the borrower.
- dti: The debt-to-income ratio of the borrower (amount of debt divided by annual income).
- fico: The FICO credit score of the borrower.
- days.with.cr.line: The number of days the borrower has had a credit line.
- revol.bal: The borrower's revolving balance (amount unpaid at the end of the credit card billing cycle).
- revol.util: The borrower's revolving line utilization rate (the amount of the credit line used relative to total credit available).
- inq.last.6mths: The borrower's number of inquiries by creditors in the last 6 months.
- delinq.2yrs: The number of times the borrower had been 30+ days past due on a payment in the past 2 years.
- pub.rec: The borrower's number of derogatory public records (bankruptcy filings, tax liens, or judgments).

Import Libraries

Import the usual libraries for pandas and plotting. You can import sklearn later on.

In [3]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

sns.set(style="whitegrid")
pd.set_option("display.max_columns", None)
```

Get the Data

** Use pandas to read loan_data.csv as a dataframe called loans.**

In [4]:
```
df = pd.read_csv('loan_data.csv')
df.head()
```

Out[4]:

	credit.policy	purpose	int.rate	installment	log.annual.inc	dti	fico	day
0	1	debt_consolidation	0.1189	829.10	11.350407	19.48	737	
1	1	credit_card	0.1071	228.22	11.082143	14.29	707	
2	1	debt_consolidation	0.1357	366.86	10.373491	11.63	682	
3	1	debt_consolidation	0.1008	162.34	11.350407	8.10	712	
4	1	credit_card	0.1426	102.92	11.299732	14.97	667	

** Check out the info(), head(), and describe() methods on loans.**

In [10]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9578 entries, 0 to 9577
Data columns (total 14 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   credit.policy      9578 non-null   int64  
 1   purpose            9578 non-null   str    
 2   int.rate           9578 non-null   float64
 3   installment        9578 non-null   float64
 4   log.annual.inc     9578 non-null   float64
 5   dti                9578 non-null   float64
 6   fico               9578 non-null   int64  
 7   days.with.cr.line  9578 non-null   float64
 8   revol.bal          9578 non-null   int64  
 9   revol.util         9578 non-null   float64
 10  inq.last.6mths     9578 non-null   int64  
 11  delinq.2yrs        9578 non-null   int64  
 12  pub.rec            9578 non-null   int64  
 13  not.fully.paid     9578 non-null   int64  
dtypes: float64(6), int64(7), str(1)
memory usage: 1.2 MB
```

In [8]:
```
df.describe()
```

Out[8]:

	credit.policy	int.rate	installment	log.annual.inc	dti	fico
count	9578.000000	9578.000000	9578.000000	9578.000000	9578.000000	9578.000000
mean	0.804970	0.122640	319.089413	10.932117	12.606679	710.846314
std	0.396245	0.026847	207.071301	0.614813	6.883970	37.970537
min	0.000000	0.060000	15.670000	7.547502	0.000000	612.000000
25%	1.000000	0.103900	163.770000	10.558414	7.212500	682.000000
50%	1.000000	0.122100	268.950000	10.928884	12.665000	707.000000
75%	1.000000	0.140700	432.762500	11.291293	17.950000	737.000000
max	1.000000	0.216400	940.140000	14.528354	29.960000	827.000000

In [9]:
```
df.head()
```

Out[9]:

	credit.policy	purpose	int.rate	installment	log.annual.inc	dti	fico	day
0	1	debt_consolidation	0.1189	829.10	11.350407	19.48	737	
1	1	credit_card	0.1071	228.22	11.082143	14.29	707	
2	1	debt_consolidation	0.1357	366.86	10.373491	11.63	682	
3	1	debt_consolidation	0.1008	162.34	11.350407	8.10	712	
4	1	credit_card	0.1426	102.92	11.299732	14.97	667	

Exploratory Data Analysis

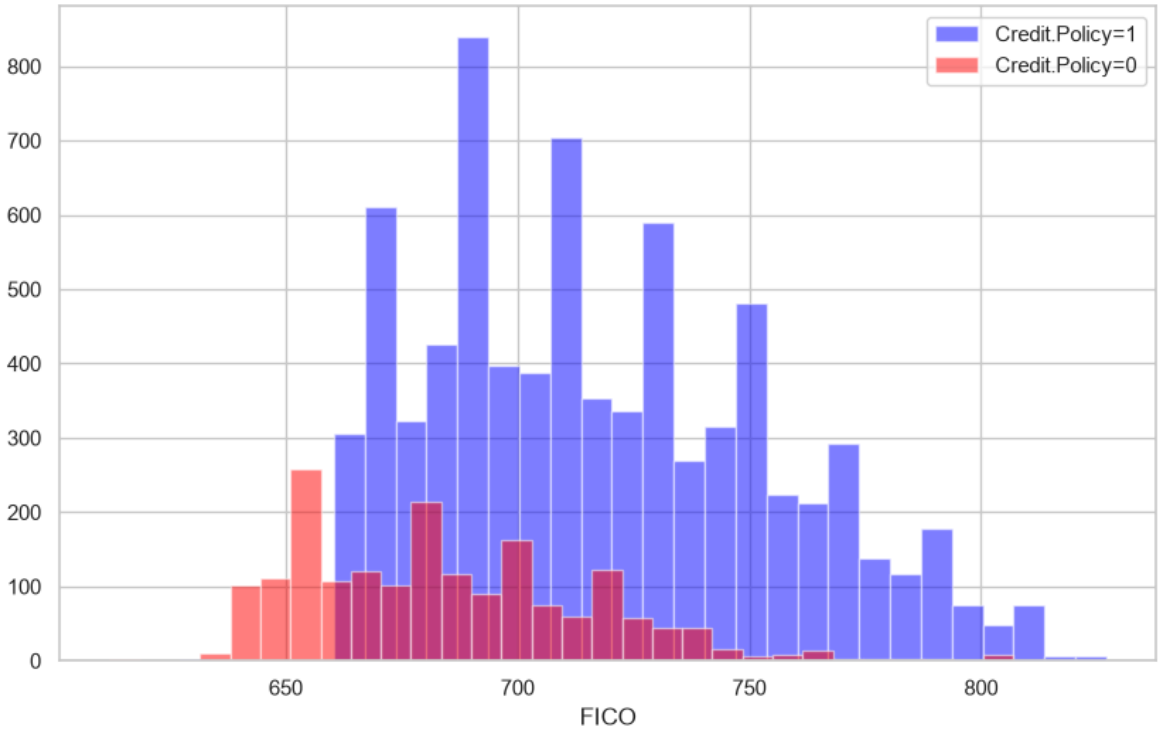
Let's do some data visualization! We'll use seaborn and pandas built-in plotting capabilities, but feel free to use whatever library you want. Don't worry about the colors matching, just worry about getting the main idea of the plot.

** Create a histogram of two FICO distributions on top of each other, one for each credit.policy outcome.**

Note:You'll probably need one line of code for each histogram, I also recommend just using pandas built in .hist()

In [11]:
```
plt.figure(figsize=(10,6))
df[df['credit.policy']==1]['fico'].hist(alpha=0.5, color='blue', bins=30, label=
df[df['credit.policy']==0]['fico'].hist(alpha=0.5, color='red', bins=30, label='
plt.legend()
plt.xlabel('FICO')
```

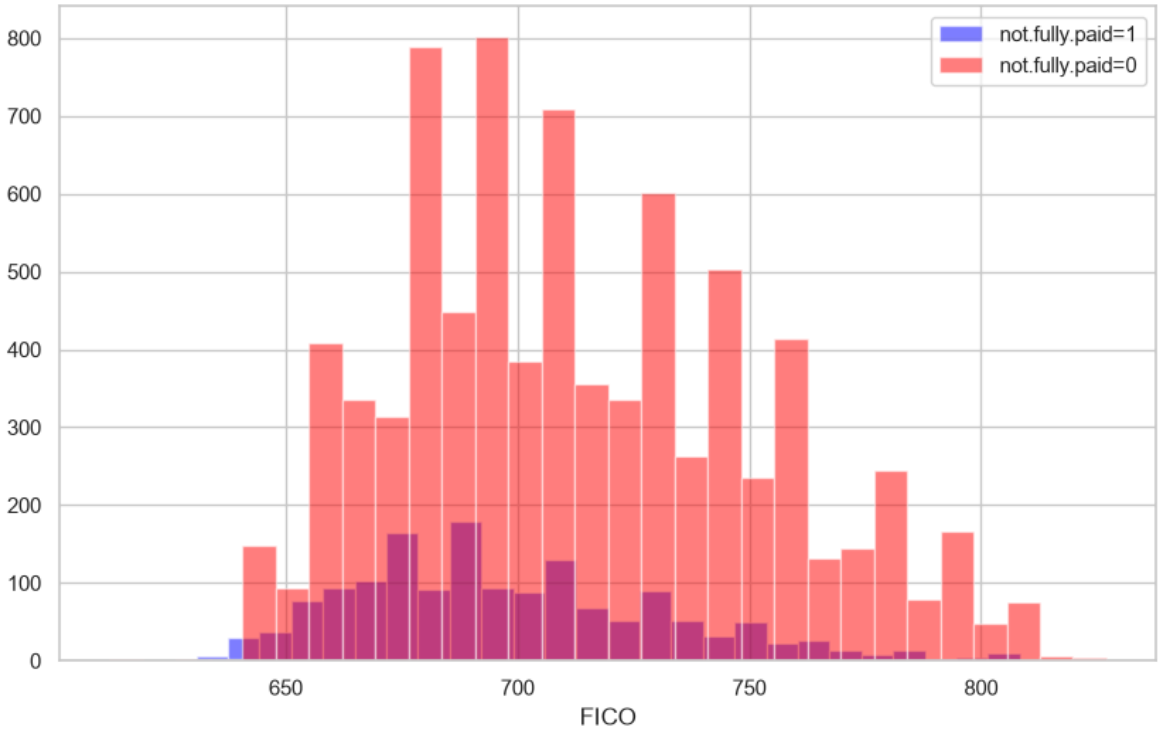
Out[11]:
```
Text(0.5, 0, 'FICO')
```



** Create a similar figure, except this time select by the not.fully.paid column.**

In [12]:
```
plt.figure(figsize=(10,6))
df[df['not.fully.paid']==1]['fico'].hist(alpha=0.5, color='blue', bins=30, label
df[df['not.fully.paid']==0]['fico'].hist(alpha=0.5, color='red', bins=30, label=
plt.legend()
plt.xlabel('FICO')
```

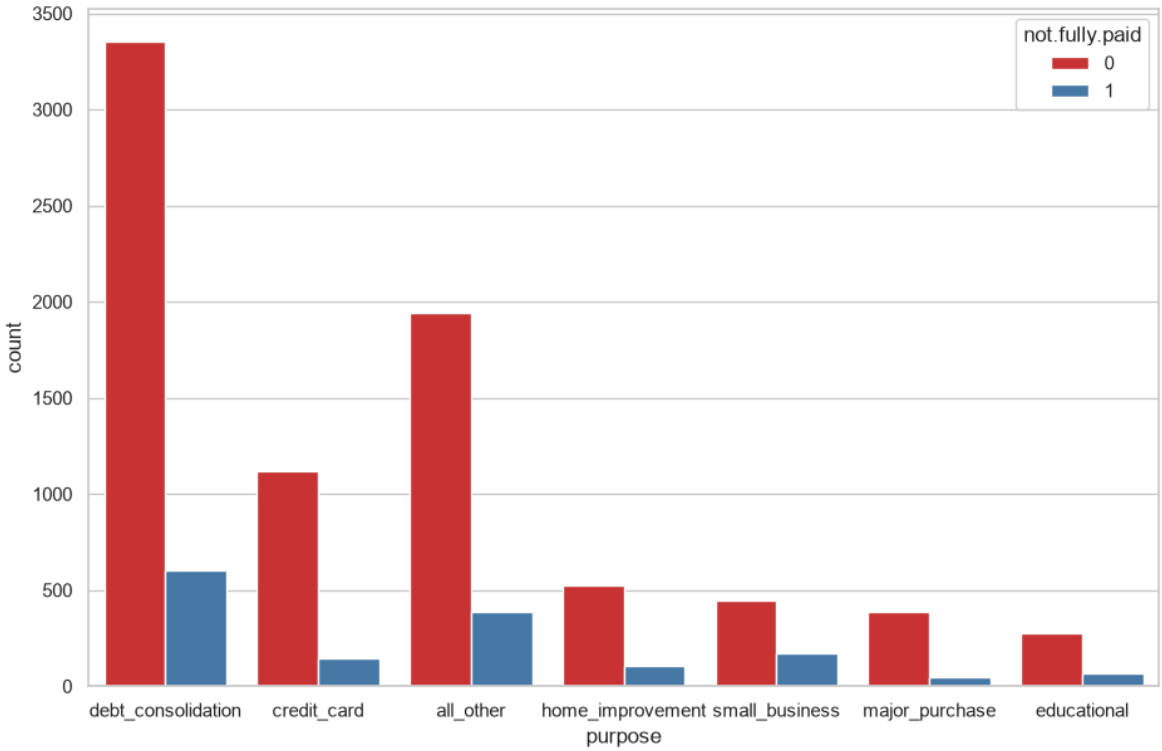
Out[12]:
```
Text(0.5, 0, 'FICO')
```



** Create a countplot using seaborn showing the counts of loans by purpose, with the color hue defined by not.fully.paid. **

In [13]:
```
plt.figure(figsize=(11,7))
sns.countplot(x='purpose', hue='not.fully.paid', data=df, palette='Set1')
```

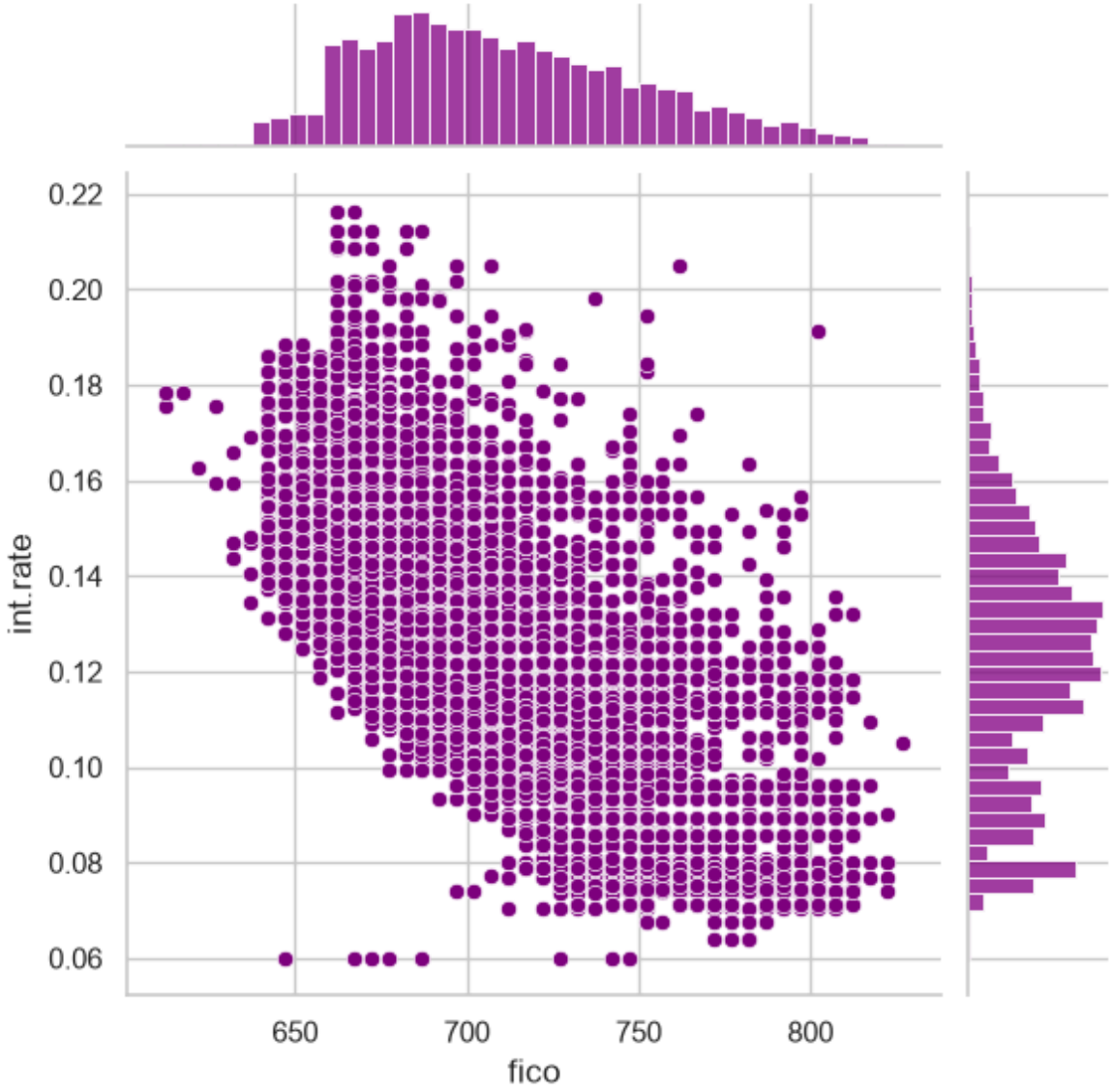
Out[13]:
```
<Axes: xlabel='purpose', ylabel='count'>
```



** Let's see the trend between FICO score and interest rate. Recreate the following jointplot.**

In [14]:
```
sns.jointplot(x='fico', y='int.rate', data=df, color='purple')
```

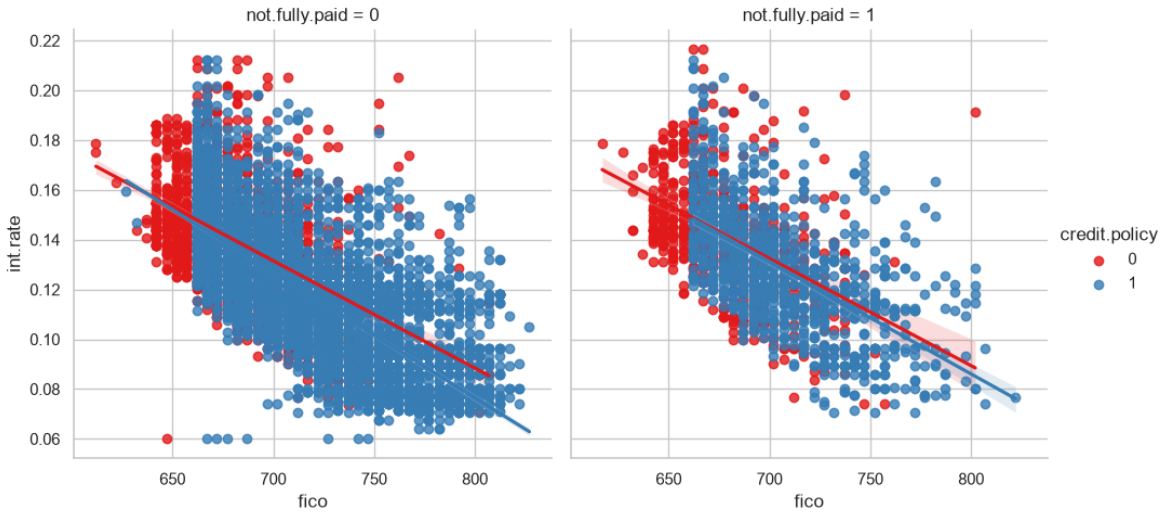
Out[14]:
```
<seaborn.axisgrid.JointGrid at 0x1c5fa2034d0>
```



** Create the following lmplots to see if the trend differed between not.fully.paid and credit.policy. Check the documentation for lmplot() if you can't figure out how to separate it into columns.**

In [15]:
```
sns.lmplot(y='int.rate', x='fico', data=df, hue='credit.policy', col='not.fully.
```

Out[15]:
```
<seaborn.axisgrid.FacetGrid at 0x1c5fad586e0>
```



Setting up the Data

Let's get ready to set up our data for our Random Forest Classification Model!

Check loans.info() again.

In [16]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9578 entries, 0 to 9577
Data columns (total 14 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   credit.policy      9578 non-null   int64  
 1   purpose            9578 non-null   str    
 2   int.rate           9578 non-null   float64
 3   installment        9578 non-null   float64
 4   log.annual.inc     9578 non-null   float64
 5   dti                9578 non-null   float64
 6   fico               9578 non-null   int64  
 7   days.with.cr.line  9578 non-null   float64
 8   revol.bal          9578 non-null   int64  
 9   revol.util         9578 non-null   float64
 10  inq.last.6mths     9578 non-null   int64  
 11  delinq.2yrs        9578 non-null   int64  
 12  pub.rec            9578 non-null   int64  
 13  not.fully.paid     9578 non-null   int64  
dtypes: float64(6), int64(7), str(1)
memory usage: 1.2 MB
```

Categorical Features

Notice that the **purpose** column as categorical

That means we need to transform them using dummy variables so sklearn will be able to understand them. Let's do this in one clean step using pd.get_dummies.

Let's show you a way of dealing with these columns that can be expanded to multiple categorical features if necessary.

Create a list of 1 element containing the string 'purpose'. Call this list cat_feats.

In [17]:
```
cat_feats = ['purpose']
```

Now use pd.get_dummies(loans,columns=cat_feats,drop_first=True) to create a fixed larger dataframe that has new feature columns with dummy variables. Set this dataframe as final_data.

In [18]:
```
final_data = pd.get_dummies(df, columns=cat_feats, drop_first=True)
```

In [ ]:
```

```

Train Test Split

Now its time to split our data into a training set and a testing set!

** Use sklearn to split your data into a training set and a testing set as we've done in the past.**

In [26]:
```
from sklearn.model_selection import train_test_split

X = final_data.drop('not.fully.paid', axis=1)
y = final_data['not.fully.paid']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random
```

Training a Decision Tree Model

Let's start by training a single decision tree first!

** Import DecisionTreeClassifier**

In [27]:
```
from sklearn.tree import DecisionTreeClassifier
```

Create an instance of DecisionTreeClassifier() called dtree and fit it to the training data.

In [28]:
```
dtree = DecisionTreeClassifier()
```

In [29]:
```
dtree.fit(X_train, y_train)
```

Out[29]:

▾ DecisionTreeClassifier

▸ Parameters

▸ Fitted attributes

Predictions and Evaluation of Decision Tree

Create predictions from the test set and create a classification report and a confusion matrix.

In [30]:
```
predictions = dtree.predict(X_test)
```

In [31]:
```
from sklearn.metrics import classification_report, confusion_matrix
```

In [32]:
```
print(classification_report(y_test, predictions))
```

```
                precision    recall  f1-score   support

           0       0.86      0.82      0.84      2431
           1       0.20      0.24      0.22       443

    accuracy                           0.73      2874
   macro avg       0.53      0.53      0.53      2874
weighted avg       0.75      0.73      0.74      2874
```

In [36]:
```
print(confusion_matrix(y_test, predictions))
```

```
[[1992  439]
 [ 336  107]]
```

Training the Random Forest model

Now its time to train our model!

Create an instance of the RandomForestClassifier class and fit it to our training data from the previous step.

In [37]:
```
from sklearn.ensemble import RandomForestClassifier
```

In [38]:
```
rfc = RandomForestClassifier(n_estimators=600)
```

In [39]:
```
rfc.fit(X_train, y_train)
```

Out[39]:

▾ RandomForestClassifier

▸ Parameters

▸ Fitted attributes

Predictions and Evaluation

Let's predict off the y_test values and evaluate our model.

** Predict the class of not.fully.paid for the X_test data.**

In [41]:
```
predictions = rfc.predict(X_test)
```

Now create a classification report from the results. Do you get anything strange or some sort of warning?

In [42]:
```
from sklearn.metrics import classification_report,confusion_matrix
```

In [43]:
```
print(classification_report(y_test,predictions))
```

```
              precision    recall  f1-score   support

           0       0.85      1.00      0.92      2431
           1       0.53      0.02      0.03       443

    accuracy                           0.85      2874
   macro avg       0.69      0.51      0.48      2874
weighted avg       0.80      0.85      0.78      2874
```

Show the Confusion Matrix for the predictions.

In [44]:
```
print(confusion_matrix(y_test,predictions))
```

```
[[2424    7]
 [ 435    8]]
```

What performed better the random forest or the decision tree?

It depends on the evaluation metric:

Overall Accuracy: The Random Forest performed better with an overall accuracy of 85% compared to 73% for the Decision Tree.

Recall for Class 1 (Borrowers not fully paying): The Decision Tree performed better with a recall of 24% (107 correctly identified cases) compared to only 2% (8 cases) for the Random Forest.

Conclusion: Neither model is fully optimized due to class imbalance, but Random Forest is overall better in accuracy, while Decision Tree is better at capturing risky borrowers (Class 1).

Random Forest:

Accuracy: 85%

Recall (Class 1): 0.02 (Correctly identified only 8 cases out of 443)

Decision Tree:

Accuracy: 73%

Recall (Class 1): 0.24 (Correctly identified 107 cases out of 443)

Great Job!